Section 3.5: Adam

A Smarter Way to Update Weights

AI Class — Lecture Notes

---

1 Two Problems with Plain Gradient Descent

Plain gradient descent works, but it has two frustrating properties in practice.

Problem 1 — One learning rate for every weight

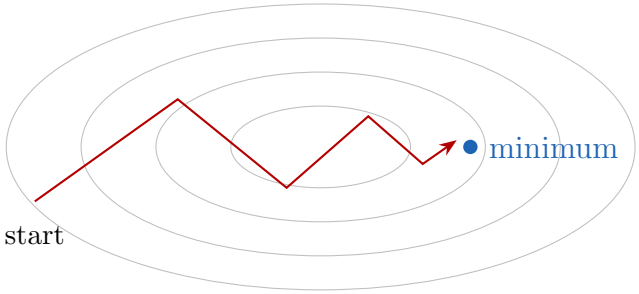
The same η is applied to every weight, regardless of how large or small its gradient is. Consider two weights in the same network:

Weight	Gradient g	GD step ($\eta = 0.1$)	Problem
w_A	-0.002	$+0.0002$	Barely moves — too slow
w_B	-5.000	$+0.500$	May overshoot — too fast

A single global η that is safe for w_B is painfully slow for w_A, and one that is fast enough for w_A will cause w_B to diverge.

Problem 2 — Oscillation in steep directions

If the loss surface is much steeper in one direction than another, gradient descent zigzags across the steep axis while crawling along the shallow one:



Adam (Adaptive Moment Estimation) fixes both problems with two ideas: momentum and adaptive step sizes.

2 Idea 1 — Momentum (the m term)

Instead of using the raw gradient at each step, keep a **running average** of recent gradients, called the **first moment** m_t:

$$m_t = \beta_1 \, m_{t-1} + (1 - \beta_1) \, g_t$$

where $g_t = \partial J / \partial w$ at step t, and $\beta_1 = 0.9$ by default.

Think of it as a weighted memory: m_t is 90% of what was accumulated before, plus 10% of the new gradient.

Situation	Effect on m_t
Gradient keeps the same sign (steady slope)	m_t grows large $\Rightarrow$ the update *accelerates*, like a ball rolling downhill.
Gradient alternates sign (oscillation)	Positive and negative values cancel out $\Rightarrow$ m_t stays small, damping the zigzag.

3 Idea 2 — Adaptive Step Sizes (the v term)

Keep a running average of the **squared** gradient, called the **second moment** v_t:

$$v_t = \beta_2 \, v_{t-1} + (1 - \beta_2) \, g_t^2$$

with $\beta_2 = 0.999$ by default. Because we square g_t, the sign is gone — v_t measures *how large* gradients have been recently.

The update then **divides by** $\sqrt{v_t}$:

History of g	v_t	Effective step
Consistently large	Large	$\propto 1/\sqrt{\text{large}} = \text{small} \Rightarrow$ pulls back weights with big gradients
Consistently small	Small	$\propto 1/\sqrt{\text{small}} = \text{large} \Rightarrow$ pushes forward weights with tiny gradients

Every weight gets its own *effective* learning rate, automatically tuned by its recent gradient history.

4 The Adam Update Rule

Combining momentum and adaptive step sizes, and adding a small **bias correction** (explained below):

At each training step t, for each weight w with gradient g_t:

Update the running averages:

$$m_t = \beta_1\, m_{t-1} + (1-\beta_1)\, g_t$$
$$v_t = \beta_2\, v_{t-1} + (1-\beta_2)\, g_t^2$$

Apply bias correction:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

Update the weight:

$$w \longleftarrow w - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

Default hyperparameters: $\eta = 0.001, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \varepsilon = 10^{-8}$.

ε is a tiny constant added to prevent division by zero when $\hat{v}_t \approx 0$.

Why bias correction? At the start ($t = 1$), $m_0 = 0$ and $v_0 = 0$ — the running averages are initialised at zero and are therefore biased too low. Dividing by $(1-\beta^t)$ compensates: at $t = 1$ it divides by $(1 - 0.9) = 0.1$, scaling the average back up to reflect the single gradient seen so far. After many steps $\beta^t \approx 0$, so the correction disappears and the raw averages are used directly.

5 Hand-Calculated Example

We use the gradient $g_1 = \partial J/\partial w_1^{(2)} = -0.084$ computed in Section 3.4 for the first training step ($t = 1$). Both running averages start at zero: $m_0 = 0$, $v_0 = 0$.

Step 1 Update the running averages

$$m_1 = 0.9 \times 0 + 0.1 \times (-0.084) = -0.0084$$
$$v_1 = 0.999 \times 0 + 0.001 \times (-0.084)^2 = 0.001 \times 0.007056 = 7.056 \times 10^{-6}$$

Step 2 Apply bias correction ($t = 1$)

$$\hat{m}_1 = \frac{m_1}{1-0.9^1} = \frac{-0.0084}{0.1} = -0.084$$
$$\hat{v}_1 = \frac{v_1}{1-0.999^1} = \frac{7.056 \times 10^{-6}}{0.001} = 7.056 \times 10^{-3}$$

Step 3 Update the weight

$$\sqrt{\hat{v}_1} = \sqrt{7.056 \times 10^{-3}} = 0.084$$

$$w_1^{(2)} \longleftarrow 0.5 - 0.001 \times \frac{-0.084}{0.084 + 10^{-8}} \approx 0.5 - 0.001 \times (-1.000) = 0.501$$

A clean pattern at $t = 1$. The bias correction at the first step recovers $\hat{m}_1 = g_1$ (the raw gradient) and $\sqrt{\hat{v}_1} = |g_1|$ (the absolute gradient). The effective step is therefore:

$$\eta \times \frac{g_1}{|g_1|} = \eta \times \text{sign}(g_1) = \pm 0.001.$$

At step 1, Adam always takes a step of exactly $\pm\eta$, regardless of how large or small the gradient is. The gradient magnitude starts shaping the step only as v_t accumulates history over multiple steps.

Comparison with plain gradient descent

	Plain GD ($\eta = 0.1$)	Adam ($\eta = 0.001$)
Gradient g_1	-0.084	-0.084
Weight before	0.500	0.500
Update size	$0.1 \times 0.084 = 0.0084$	0.001
Weight after	0.508	0.501

The step sizes differ because the two methods use different η values. The key advantage of Adam is not the size of this first step but that the effective step size *adapts* to each weight's gradient history — it neither stalls on weights with tiny gradients nor explodes on weights with large ones.

6 Why Adam Is the Default Choice

Property	Plain GD	Adam
Learning rate to tune	One global η	One global η (but less sensitive)
Per-weight step size	Same for all	Adapts to gradient history
Handles oscillation	No	Yes (momentum cancels zigzags)
Handles sparse gradients	Poorly	Well (small $v \to$ large effective step)
Default η	Problem-specific	0.001 usually works

Adam is not magic. It still requires choosing η, and it can overfit faster than plain GD in some settings. For problems where generalisation matters most, simpler optimisers (SGD with momentum) are sometimes preferred. Adam is, however, the best default starting point for most deep learning applications.

7 The Complete Training Loop

Putting everything from Sections 3.2–3.5 together:

Step	Action
1	Initialise all weights randomly; set $m = 0$, $v = 0$, $t = 0$.
2	For each training example (or mini-batch):
	(a) **Forward pass** — compute $\hat{y}$ and J.
	(b) **Backpropagation** — compute $\partial J/\partial w$ for every weight.
	(c) **Adam update** — update m, v, then w.
	(d) Increment t.
3	Repeat Step 2 until J is small enough or a maximum number of epochs is reached.